



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 3
по дисциплине «Анализ Алгоритмов»

Тема Поиск в массиве

Студент Поляков А.И.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Алгоритм поиска перебором	4
1.2 Алгоритм бинарного поиска	4
1.3 Вывод	4
2 Конструкторская часть	5
2.1 Требования к программному обеспечению	5
2.2 Вариации бинарного поиска	5
2.3 Разработка алгоритмов	6
2.4 Вывод	7
3 Технологическая часть	8
3.1 Средства разработки	8
3.2 Реализация алгоритмов	8
3.3 Вывод	9
4 Исследовательская часть	10
4.1 Технические характеристики	10
4.2 Расчёт размера массива	10
4.3 Трудоёмкость алгоритмов	10
4.4 Вывод	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14

ВВЕДЕНИЕ

Целью данной лабораторной работы является проведение исследования трудоемкости алгоритмов поиска в массиве. Будут рассмотрены алгоритм поиска перебором и алгоритм бинарного поиска.

Задачи работы:

- Описание алгоритмов поиска в массиве полным перебором и бинарного поиска;
- Проектирование необходимых алгоритмов;
- Реализация программного обеспечения;
- Сравнительный анализ алгоритмов по трудоемкости.

1 Аналитическая часть

В данной лабораторной работе рассматриваются два алгоритма поиска в массиве: алгоритм **поиска перебором** и алгоритм **бинарного поиска**.

1.1 Алгоритм поиска перебором

Линейный поиск (полным перебором) — алгоритм, который последовательно просматривает элементы массива. Поиск прекращается, если искомый элемент найден по i -му индексу или просмотрен весь массив, и элемент не найден.

Линейный поиск универсален, так как не накладывает требований к массиву, но имеет сложность $O(N)$, что делает его не самым эффективным решением. [3]

1.2 Алгоритм бинарного поиска

Бинарный поиск — алгоритм поиска в отсортированном массиве, который последовательно делит массив пополам для нахождения искомого элемента. Поиск прекращается при выполнении одного из двух условий: искомый элемент найден по i -му индексу или диапазон поиска пуст (нижний индекс превышает верхний).

Бинарный поиск требует, чтобы массив был отсортирован, и имеет сложность $O(\log N)$, что делает его более эффективным по сравнению с линейным поиском. [3]

1.3 Вывод

В результате аналитического раздела были рассмотрены алгоритмы линейного поиска и бинарного поиска в массивах.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемой программе предъявлен ряд требований:

Входные данные: Массив целых чисел, искомое целое число.

Выходные данные: Индекс искомого числа в массиве.

- Индексация элементов в массиве начинается с 0;
- В случае, если искомого элемента в массиве нет, вместо индекса должно возвращаться значение -1.

2.2 Вариации бинарного поиска

Алгоритм бинарного поиска можно реализовать двумя способами: итеративно и рекурсивно.

В итеративной версии используются две переменные для определения левой и правой границ. В цикле вычисляется средний элемент, и границы обновляются в зависимости от его значения.

В рекурсивной версии определяется средний элемент всего массива, и в зависимости от его значения рекурсивно вызывается поиск либо в правой, либо в левой части массива.

В данной работе будет рассматриваться итеративная реализация алгоритма.

2.3 Разработка алгоритмов

Схема алгоритма поиска в массиве полным перебором представлена на рисунке 2.1.

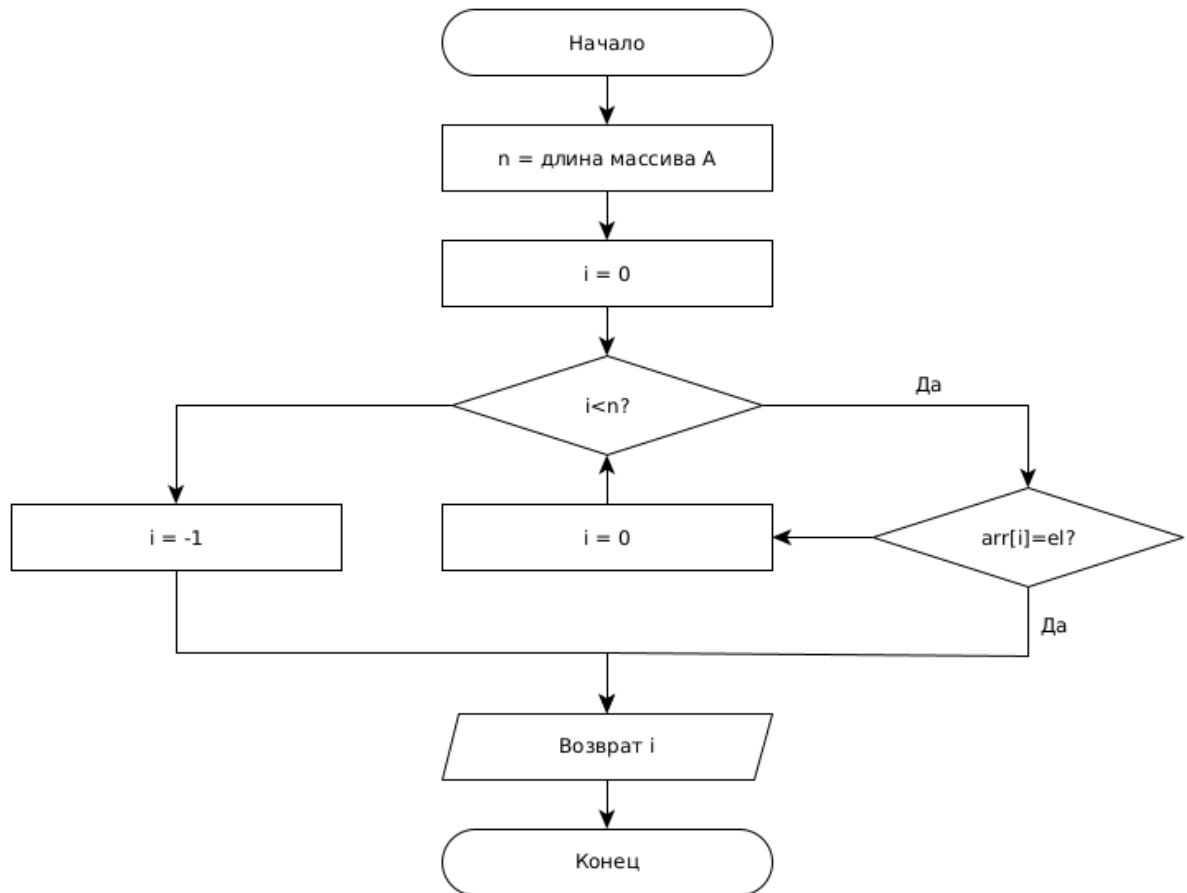


Рисунок 2.1 — Схема алгоритма поиска в массиве полным перебором

Схема алгоритма бинарного поиска представлена на рисунке 2.2.

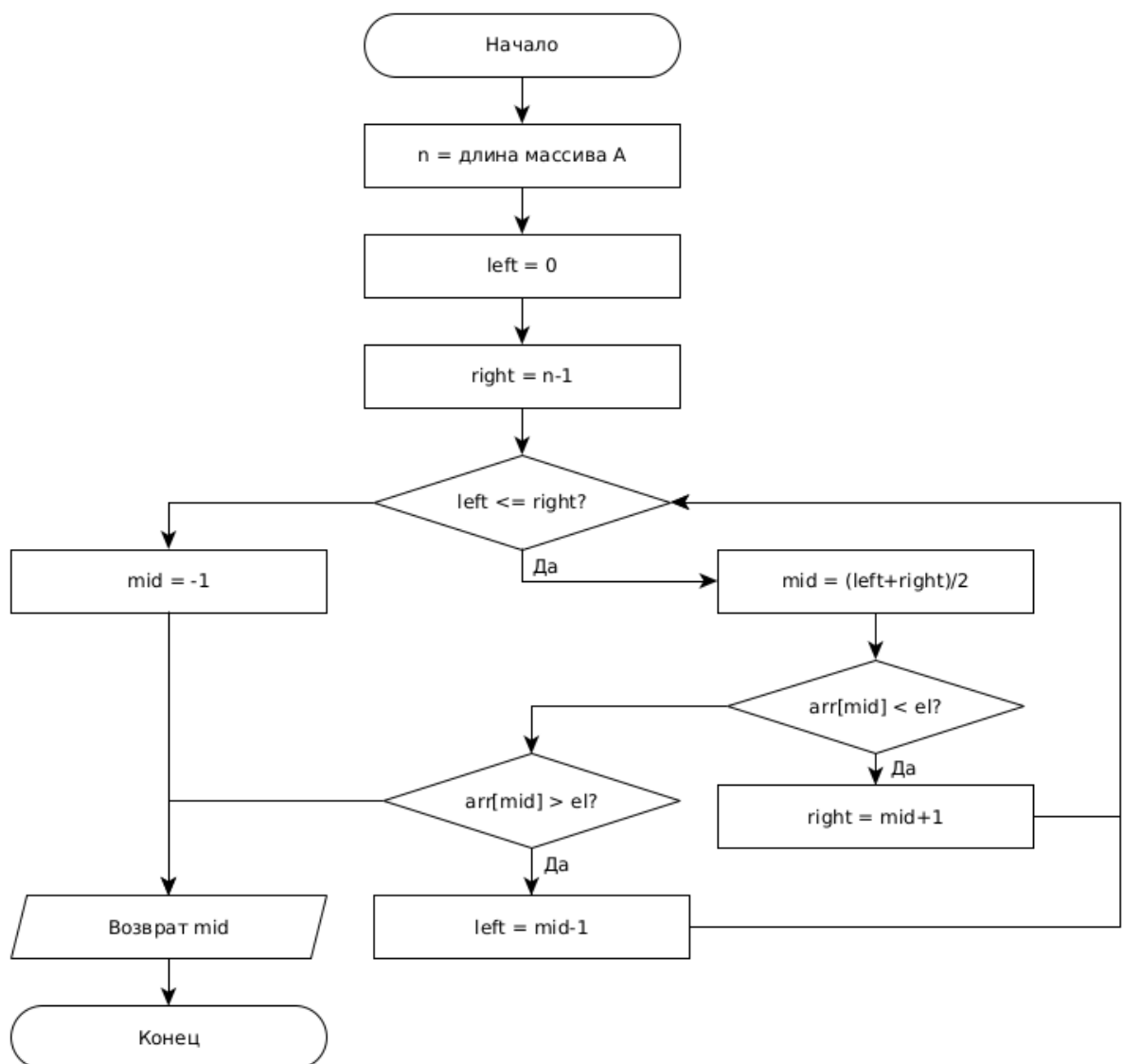


Рисунок 2.2 — Схема алгоритма бинарного поиска

2.4 Вывод

В результате конструкторской части были определены требования к ПО, спроектированы алгоритмы поиска в массиве полным перебором и бинарного поиска.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран python3 [1], так как в его стандартной библиотеке присутствуют функции замера процессорного времени и использованной памяти, которые требуются в условиях, а также данный язык обладающий множеством инструментов для визуализации и работы с данными и таблицами.

Для построения графиков использовалась библиотека plotly [2].

3.2 Реализация алгоритмов

Ниже представлены реализации алгоритмов линейного поиска и бинарного поиска. Оба алгоритма включают счётчик сравнений в теле цикла: в линейном поиске он увеличивается на 1 за каждую итерацию, в то время как в бинарном поиске счётчик может увеличиваться на 1 или 2, в зависимости от сравнения текущего элемента с искомым значением.

В листинге 3.1 представлена реализация алгоритма линейного поиска.

Листинг 3.1 — Алгоритм линейного поиска

```
def FindElem(arr, el):  
    cnt = 0  
    for i in range(len(arr)):  
        cnt += 1  
        if arr[i] == el:  
            return i, cnt  
    return -1, cnt
```


В листинге 3.2 представлена реализация алгоритма бинарного поиска.

Листинг 3.2 — Алгоритм бинарного поиска

```
def BinFindElem(arr, el):
    left = 0
    right = len(arr) - 1
    cnt = 0

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] < el:
            cnt += 1
            left = mid + 1
        elif arr[mid] > el:
            right = mid - 1
            cnt += 2
        else:
            cnt += 2
            return mid, cnt

    return -1, cnt
```

3.3 Вывод

В ходе работы были разработаны алгоритмы линейного и бинарного поиска.

4 Исследовательская часть

4.1 Технические характеристики

Технические характеристики устройства, на котором проводились замеры:

- AMD Ryzen 7 5800U (16) @ 4.51 ГГц;
- Оперативная память: 16 Гб;
- Ядро ОС Linux 6.10.10-arch1-1.

При проведении замеров ноутбук был включен в сеть и были запущены только системные приложения и реализованные программы.

4.2 Расчёт размера массива

Расчёт размера массива по варианту производится по формуле:

$$N = \frac{X}{8} + \begin{cases} X \% 1000, & \text{если } \frac{X}{4} \% 10 == 0, \\ (\frac{X}{4} \% 10) * (X \% 10) + (\frac{X}{2} \% 10), & \text{иначе,} \end{cases} \quad (4.1)$$

где $X = 8114$ – номер задачи в redmine.

Для моего варианта $N = 1053$

4.3 Трудоёмкость алгоритмов

В рамках данного исследования было проанализировано количество операций сравнения в циклах, зависящее от индекса искомого элемента в массиве. Результаты представлены в виде гистограмм: для линейного поиска — рисунок 4.1, для бинарного поиска — рисунки 4.2 и 4.3. На последнем графике столбцы упорядочены по высоте. Индекс -1 на всех графиках указывает на отсутствие искомого элемента в массиве.

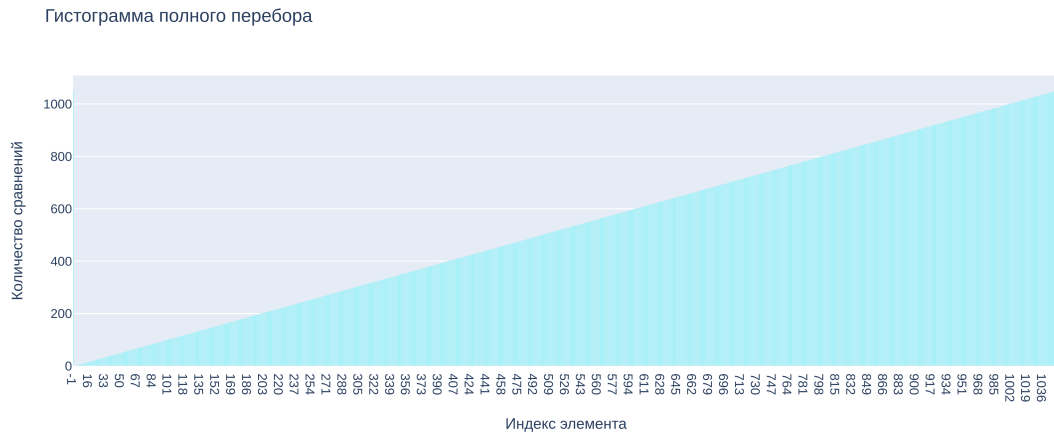


Рисунок 4.1 — Гистограмма зависимости числа сравнений от индекса элемента в линейном поиске

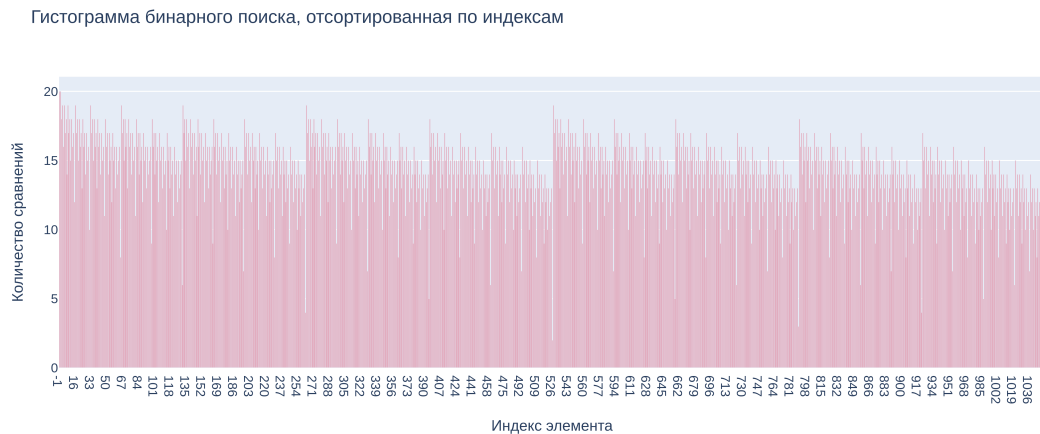


Рисунок 4.2 — Гистограмма зависимости числа сравнений от индекса элемента в бинарном поиске



Рисунок 4.3 — Гистограмма зависимости числа сравнений от индекса элемента в бинарном поиске, отсортированная по количеству сравнений

4.4 Вывод

Был проведен анализ трудоемкости алгоритмов поиска в массиве.

Как видно из рисунков 4.1–4.3, линейный поиск значительно уступает бинарному по количеству операций сравнения. При размере массива в 1053 элементов максимальное количество сравнений для линейного поиска составляет 1053, тогда как для бинарного — всего 20.

Кроме того, как отмечается на рисунке 4.3, кривая трудоемкости бинарного поиска выглядит как логарифмическая прямая, что подтверждает его логарифмическую сложность $O(\log N)$.

ЗАКЛЮЧЕНИЕ

В ходе данной лабораторной работы было проведено исследование трудоемкости алгоритмов поиска в массиве.

Были выполнены следующие задачи:

- Описаны алгоритмы поиска в массиве полным перебором и бинарного поиска;
- Спроектированы необходимые алгоритмы;
- Реализовано программное обеспечение;
- Проведен сравнительный анализ алгоритмов по трудоемкости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Python / [Электронный ресурс] // Python : [сайт]. — URL: <https://www.python.org/> (дата обращения: 24.09.2024).
2. Plotly Open Source Graphing Library for Python / [Электронный ресурс] // Plotly : [сайт]. — URL: <https://plotly.com/python/> (дата обращения: 20.09.2024).
3. Knuth, Donald (1998). Sorting and searching. The Art of Computer Programming. Vol. 3 (2nd ed.). Reading, MA: Addison-Wesley Professional. ISBN 978-0-201-89685-5.